

AI 여행 일정 플래너 MVP 개발 명세

1. 프로젝트 택하면 관광지, 맛집, 숙소, 이동 방법, 예상 비용, 지도 경로가 포함된 상세 여행 일정을 생성하는 웹서비스를 개발한다.

사용자는 생성된 일정에 대해 자연어로 수정 요청을 보낼 수 있다.

예시:

- “2일 차 일정이 너무 빡빡해. 관광지 하나를 빼줘.”
- “숙소는 유지하고 맛집만 현지인 맛집 위주로 바꿔줘.”
- “유니버설 스튜디오는 반드시 유지해줘.”
- “총예산을 10만 원 줄여줘.”
- “비가 온다는 가정으로 실내 일정으로 수정해줘.”

AI는 전체 일정을 처음부터 다시 만드는 것이 아니라 사용자의 요청과 기존 일정을 비교해 필요한 부분만 수정한다.

2. 프로젝트 임시 이름

프로젝트명: TripMate AI

한 줄 설명:

취향과 예산을 입력하면 예약 가능한 여행 상품과 실제 이동 경로까지 구성해 주는 AI 여행 플래너

3. MVP 범위

첫 번째 버전에서는 기능을 다음과 같이 제한한다.

- 한 번의 여행에서 도시 1개만 지원
- 여행 기간 1~7일
- 성인 여행자 중심
- 실제 결제 및 예약 기능은 구현하지 않음
- 외부 예약 페이지로 이동하는 링크만 제공
- 여행 초안은 3개 제공
- 선택한 초안 하나만 상세 검색
- 숙소는 3개 후보를 보여주고 사용자가 하나를 선택
- 장소와 맛집은 실제 검색 결과를 사용
- 지도에서 날짜별 경로 표시
- AI 채팅을 통한 일정 수정 지원
- 로그인, 일정 저장, 공유 링크 지원

초기 버전에서는 다중 도시 여행, 항공권 결제, 공동 편집, 실시간 가격 알림 기능을 제외한다.

4. 사용자 입력 항목

여행 생성 화면은 한 번에 긴 양식을 보여주지 않고 단계별 선택형 마법사로 구성한다.

1단계: 기본 여행 정보

- 출발 지역
- 여행 도시
- 여행 시작일
- 여행 종료일
- 인원수
- 동행 유형
- 혼자
- 연인
- 친구
- 가족
- 부모님
- 아이 동반
- 기타

2단계: 여행 취향

복수 선택 가능:

- 유명 관광지
- 맛집 탐방
- 카페
- 쇼핑
- 자연과 풍경
- 역사와 문화
- 액티비티
- 테마파크
- 휴양
- 야경
- 사진 촬영
- 현지인 코스
- 술과 밤 문화

3단계: 여행 속도

- 여유롭게
- 적당히
- 알차게

각 속도의 기본 규칙:

- 여유롭게: 하루 핵심 장소 2~3개
- 적당히: 하루 핵심 장소 3~4개
- 알차게: 하루 핵심 장소 4~6개

4단계: 예산

- 1인 기준 / 전체 기준
- 총예산
- 예산 조정 가능 범위
- 절대 초과 금지
- 최대 10% 초과 가능
- 만족도가 높으면 최대 20% 초과 가능

예산 분류:

- 항공 및 교통
- 숙소
- 관광 및 체험
- 식사
- 기타

5단계: 숙소 취향

- 가성비 우선
- 위치 우선
- 시설 우선
- 감성 숙소
- 가족형 숙소
- 호텔
- 한인민박
- 호스텔
- 상관없음

추가 조건:

- 1박 최대 금액
- 선호 지역
- 금연 여부
- 조식 필요 여부
- 역에서 도보 몇 분 이내인지

6단계: 추가 조건

- 반드시 방문할 장소
 - 제외할 장소
 - 먹지 못하는 음식 및 알레르기
 - 많이 걷기 어려움
 - 유모차 또는 휠체어 이용
 - 선호 이동 수단
 - 자유 입력 요청
-

5. 전체 사용자 흐름

STEP 1. 조건 입력

사용자가 여행 조건을 선택하고 “여행 초안 만들기” 버튼을 누른다.

STEP 2. 초안 3개 생성

AI는 상세 장소를 모두 검색하기 전에 서로 다른 성격의 초안을 생성한다.

예시:

플랜 A: 필수 관광지 집중형

- 대표 관광지를 빠짐없이 방문
- 이동량이 비교적 많음
- 예상 비용 82만 원

플랜 B: 맛집과 감성 중심형

- 관광지 수를 줄이고 맛집과 카페 비중 확대
- 여유로운 일정
- 예상 비용 76만 원

플랜 C: 가성비 현지인 코스

- 무료 관광지와 대중교통 중심
- 현지인 방문 지역 포함
- 예상 비용 64만 원

각 초안 카드에는 다음을 표시한다.

- 플랜 이름
- 한 줄 설명
- 여행 스타일
- 하루별 핵심 지역
- 대표 활동
- 대략적인 예상 비용
- 장점
- 단점
- 예상 이동량
- 사용자 조건 적합도

STEP 3. 초안 선택

사용자가 초안 하나를 선택한다.

이 시점부터 다음 작업을 수행한다.

1. 실제 관광지 검색
2. 실제 맛집 검색

3. 숙소 후보 검색
4. 마이리얼트립 상품 검색
5. 각 장소의 위치 확인
6. 장소 간 이동시간 계산
7. 날짜별 동선 최적화
8. 상세 예산 계산

STEP 4. 상세 일정 표시

상세 일정 화면에는 다음을 표시한다.

- 날짜별 타임라인
- 관광지
- 식사 장소
- 카페
- 숙소
- 이동 수단
- 이동시간
- 예상 비용
- 예약 상품
- 지도 경로
- 일정 수정 채팅

STEP 5. AI 수정

사용자가 자연어로 수정사항을 입력한다.

AI는 수정사항을 분석하여 다음 작업 중 하나 이상을 수행한다.

- 일정 추가
- 일정 삭제
- 일정 교체
- 일정 시간 변경
- 날짜 변경
- 식당 변경
- 숙소 변경
- 이동 수단 변경
- 예산 조정
- 하루 전체 재구성

수정 후에는 반드시 이동시간과 예상 비용을 다시 계산한다.

6. 핵심 화면 구성

화면 1: 랜딩 페이지

구성:

- 서비스 소개 문구
- 여행 생성 버튼

- 예시 여행 일정
- 주요 기능 소개

메인 문구:

여행 조건만 골라주세요.
 AI가 일정부터 맛집, 숙소, 이동 경로까지 준비해 드립니다.

화면 2: 여행 조건 입력

- 상단 진행률 표시
- 이전/다음 버튼
- 다중 선택 카드
- 날짜 선택기
- 예산 슬라이더
- 추가 요청 입력창

화면 3: 여행 초안 비교

3개의 플랜 카드를 가로로 보여준다.

비교 항목:

- 가격
- 여행 속도
- 관광 비중
- 맛집 비중
- 이동량
- 휴식 시간
- 추천 대상

사용자는 “이 플랜으로 상세 일정 만들기”를 선택한다.

화면 4: 상세 플래너

데스크톱 기준 3개 영역으로 구성한다.

왼쪽: 날짜별 일정

- 날짜 탭
- 시간순 일정 카드
- 장소 사진
- 장소명
- 방문 시간
- 체류 시간
- 예상 비용
- 이동 방법

중앙: Google 지도

- 일정 장소 마커
- 방문 순서 번호

- 날짜별 이동 경로
- 도보, 대중교통, 차량 경로
- 마커 클릭 시 장소 정보

오른쪽: 여행 정보 패널

탭 구성:

- AI 수정
- 예산
- 숙소
- 예약 상품

모바일에서는 다음 탭으로 분리한다.

- 일정
- 지도
- 예산
- AI 수정

7. 추천 기술 스택

프론트엔드

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- React Hook Form
- Zod
- TanStack Query

백엔드

- Next.js Route Handler
- 복잡한 MCP 연결이 필요하면 별도의 Node.js 백엔드 사용
- Supabase PostgreSQL
- Supabase Auth

AI

- 구조화된 JSON 출력을 지원하는 LLM
- 일정 생성용 프롬프트
- 일정 수정용 프롬프트
- 장소 평가 및 순위 지정용 프롬프트

외부 데이터

- 마이리얼트립 MCP
- Google Places API
- Google Maps JavaScript API

- Google Routes API

8. 외부 서비스 역할 분리

마이리얼트립 MCP

다음과 같은 실제 여행 상품 검색에 사용한다.

- 투어
- 티켓
- 액티비티
- 교통 상품
- 항공권
- 숙소

단, 실제 사용할 수 있는 상품 종류와 검색 조건은 마이리얼트립 MCP가 제공하는 도구 목록을 조회한 후 구현한다.

마이리얼트립 MCP 응답은 내부 공통 형식으로 변환한다.

```
type TravelProduct = {  
  id: string;  
  category: "flight" | "hotel" | "activity" | "ticket" | "transport";  
  title: string;  
  price: number | null;  
  currency: string;  
  imageUrl: string | null;  
  bookingUrl: string;  
  locationName: string | null;  
  availableDate: string | null;  
  source: "myrealtrip";  
};
```

Google Places API

다음 정보를 검색한다.

- 관광지
- 맛집
- 카페
- 쇼핑 장소
- 장소 주소
- 위도와 경도
- 평점
- 리뷰 수
- 영업시간
- 가격대
- 장소 사진
- Google 지도 링크

Google Routes API

다음 정보를 계산한다.

- 장소 간 거리
- 예상 이동시간
- 도보 경로
- 차량 경로
- 대중교통 경로
- 날짜별 전체 이동 경로

LLM

LLM은 실제 장소 정보를 직접 만들어내지 않는다.

LLM의 역할:

- 사용자 취향 분석
- 여행 콘셉트 생성
- 검색 조건 생성
- 검색 결과 순위 결정
- 일정 구성
- 수정 요청 해석
- 추천 이유 작성

장소명, 가격, 위치, 평점, 영업시간은 외부 API 결과를 기준으로 사용한다.

9. 시스템 아키텍처

```
[Next.js Frontend]
|
v
[Planner API / Backend]
|
+-----> [LLM]
|         - 사용자 조건 분석
|         - 여행 초안 생성
|         - 일정 구성
|         - 수정 명령 생성
|
+-----> [Myrealtrip MCP Adapter]
|         - 상품 검색
|
+-----> [Google Places Adapter]
|         - 관광지·맛집·숙소 검색
|
+-----> [Google Routes Adapter]
|         - 거리·시간·경로 계산
```

```
|
+-----> [Supabase]
- 사용자
- 여행
- 일정
- 수정 기록
```

프론트엔드에서 MCP나 비밀 API 키를 직접 호출하지 않는다. 모든 외부 데이터 호출은 서버를 거친다.

10. 프로젝트 폴더 구조

```
src/
├─ app/
│   ├─ page.tsx
│   ├─ trips/
│   │   ├─ new/
│   │   │   └─ page.tsx
│   │   └─ [tripId]/
│   │       └─ page.tsx
│   └─ api/
│       ├─ plans/
│       │   ├─ drafts/
│       │   │   └─ route.ts
│       │   └─ generate/
│       │       └─ route.ts
│       └─ revise/
│           └─ route.ts
│       ├─ places/
│       │   └─ search/
│       │       └─ route.ts
│       └─ routes/
│           └─ calculate/
│               └─ route.ts
│
├─ components/
│   ├─ trip-form/
│   ├─ draft-plans/
│   ├─ itinerary/
│   ├─ map/
│   ├─ budget/
│   └─ ai-chat/
│
├─ lib/
│   ├─ ai/
│   │   ├─ prompts.ts
│   │   └─ schemas.ts
```

```

| |   └─ client.ts
| |   └─ myrealtrip/
| |       └─ provider.ts
| |       └─ mock-provider.ts
| |       └─ mcp-provider.ts
| |   └─ google/
| |       └─ places.ts
| |       └─ routes.ts
| |   └─ planner/
| |       └─ build-drafts.ts
| |       └─ enrich-plan.ts
| |       └─ optimize-route.ts
| |       └─ calculate-budget.ts
| |       └─ apply-revision.ts
| |   └─ supabase/
|
└─ types/
    └─ trip.ts
    └─ itinerary.ts
    └─ product.ts

```

11. MCP 추상화 구조

마이리얼트립 MCP의 실제 도구 이름이 바뀌어도 서비스 코드 전체를 수정하지 않도록 Provider 패턴을 사용한다.

```

interface TravelProductProvider {
  searchFlights(input: FlightSearchInput): Promise<TravelProduct[]>;
  searchHotels(input: HotelSearchInput): Promise<TravelProduct[]>;
  searchActivities(input: ActivitySearchInput): Promise<TravelProduct[]>;
}

```

개발 초기에는 다음 구현을 사용한다.

```
MockMyrealtripProvider
```

실제 MCP 연결 후 다음 구현으로 교체한다.

```
MyrealtripMcpProvider
```

환경변수로 전환한다.

TRAVEL_PROVIDER=mock

또는:

TRAVEL_PROVIDER=myrealtrip

MCP에서 제공하지 않는 기능은 빈 배열을 반환하거나 Google Places 검색으로 대체한다.

12. 여행 초안 JSON 구조

```
{
  "drafts": [
    {
      "id": "draft-a",
      "title": "대표 명소 집중 플랜",
      "concept": "처음 방문하는 여행자를 위한 핵심 관광 일정",
      "summary": "대표 관광지를 중심으로 구성한 알찬 일정입니다.",
      "estimatedBudget": {
        "total": 800000,
        "currency": "KRW"
      },
      "pace": "busy",
      "walkingLevel": "high",
      "highlights": [
        "대표 관광지",
        "전망대",
        "현지 음식"
      ],
      "days": [
        {
          "day": 1,
          "area": "난바·도톤보리",
          "themes": ["맛집", "야경"]
        }
      ],
      "pros": ["대표 명소를 빠짐없이 방문"],
      "cons": ["하루 이동량이 많음"],
      "matchScore": 91
    }
  ]
}
```

이 단계에서는 장소의 상세 정보나 정확한 가격을 확정하지 않는다.

13. 상세 일정 JSON 구조

```
{
  "tripId": "trip-001",
  "title": "오사카 3박 4일 맛집·관광 여행",
  "currency": "KRW",
  "hotel": {
    "id": "hotel-01",
    "name": "숙소명",
    "placeId": "google-place-id",
    "latitude": 0,
    "longitude": 0,
    "nightlyPrice": 120000,
    "bookingUrl": "",
    "source": "myrealtrip"
  },
  "days": [
    {
      "day": 1,
      "date": "2026-08-01",
      "items": [
        {
          "id": "item-001",
          "type": "attraction",
          "title": "오사카성",
          "placeId": "google-place-id",
          "latitude": 0,
          "longitude": 0,
          "startTime": "10:00",
          "endTime": "12:00",
          "durationMinutes": 120,
          "estimatedCost": 6000,
          "source": "google_places",
          "locked": false,
          "transportFromPrevious": {
            "mode": "TRANSIT",
            "durationMinutes": 25,
            "distanceMeters": 5100
          }
        }
      ]
    }
  ],
  "budget": {
    "accommodation": 360000,
    "food": 180000,
    "activities": 100000,
  }
}
```

```

"transportation": 80000,
"other": 50000,
"total": 770000
},
"warnings": []
}

```

14. AI 일정 생성 규칙

AI는 다음 규칙을 반드시 지킨다.

1. 장소의 운영시간과 일정 시간이 충돌하지 않아야 한다.
2. 식사 시간을 고려한다.
3. 같은 날 방문 장소는 지리적으로 가까운 지역끼리 묶는다.
4. 장소 사이에 실제 이동시간을 포함한다.
5. 사용자가 선택한 여행 속도를 준수한다.
6. 총비용이 예산 범위를 넘으면 경고하거나 대체안을 제공한다.
7. 사용자의 필수 장소를 삭제하지 않는다.
8. 알레르기 및 접근성 조건을 준수한다.
9. 확인되지 않은 가격이나 운영시간을 사실처럼 말하지 않는다.
10. 예약 상품과 일반 장소 정보를 구분한다.
11. 일정이 너무 빡빡하면 사용자에게 경고한다.
12. 위치가 확인되지 않은 장소는 일정에 추가하지 않는다.

15. 일정 수정 방식

사용자가 수정 요청을 보내면 LLM은 완성된 새 일정을 바로 반환하지 않고 변경 명령을 반환한다.

```

{
  "summary": "2일 차 일정을 여유롭게 조정했습니다.",
  "operations": [
    {
      "type": "remove",
      "targetItemId": "item-203",
      "reason": "하루 이동량을 줄이기 위해 삭제"
    },
    {
      "type": "add",
      "day": 2,
      "afterItemId": "item-202",
      "searchQuery": "오사카성 근처 조용한 카페",
      "desiredDurationMinutes": 60
    }
  ],
  "preserveItemIds": ["item-201"],

```

```
"requiresRouteRecalculation": true,  
"requiresBudgetRecalculation": true  
}
```

백엔드는 이 명령을 검증한 후 실제 장소를 검색하고 일정을 수정한다.

수정 순서:

1. 사용자 수정 요청 분석
2. 잠금된 일정 확인
3. 변경 명령 생성
4. 필요한 장소 재검색
5. 이동 경로 재계산
6. 예산 재계산
7. 일정 충돌 검증
8. 변경 전후 차이 저장
9. 사용자에게 변경 내용 설명

사용자는 특정 일정에 “고정” 버튼을 사용할 수 있다.

고정된 일정은 사용자가 명시적으로 삭제를 요청하지 않는 한 AI가 변경하지 않는다.

16. 데이터베이스 테이블

profiles

- id
- nickname
- created_at

trips

- id
- user_id
- title
- destination
- start_date
- end_date
- travelers
- preferences_json
- budget_json
- selected_draft_id
- status
- created_at
- updated_at

draft_plans

- id
- trip_id

- title
- concept
- draft_json
- selected

itineraries

- id
- trip_id
- version
- itinerary_json
- created_at

revision_messages

- id
- trip_id
- role
- content
- operations_json
- created_at

saved_places

- id
- trip_id
- external_place_id
- source
- place_json

route_cache

- id
- origin_place_id
- destination_place_id
- travel_mode
- route_json
- expires_at

17. API 엔드포인트

POST /api/plans/drafts

입력 조건을 바탕으로 초안 3개를 생성한다.

POST /api/plans/generate

선택된 초안을 실제 장소, 상품, 경로 데이터로 상세화한다.

POST /api/plans/revise

현재 일정과 사용자 수정 요청을 받아 수정 명령을 생성하고 적용한다.

POST /api/places/search

Google Places를 통해 관광지, 식당, 카페를 검색한다.

POST /api/routes/calculate

일정에 포함된 장소 사이의 이동시간과 경로를 계산한다.

GET /api/trips/[tripId]

저장된 여행 전체 정보를 조회한다.

PATCH /api/trips/[tripId]/items/[itemId]

일정을 잠그거나 사용자가 직접 수정한다.

18. 캐싱 및 비용 절약

다음 데이터는 데이터베이스에 캐싱한다.

- 장소 기본 정보
- 장소 사진 URL
- 장소 좌표
- 동일 장소 간 경로
- 동일 조건의 상품 검색 결과
- AI가 생성한 초안

초안 3개를 만드는 단계에서는 Google Places와 Routes API를 대량 호출하지 않는다.

사용자가 초안을 선택한 후에만 다음 작업을 수행한다.

- 장소 상세 검색
- 식당 검색
- 상품 검색
- 전체 경로 계산

Google Places 요청 시 화면에 필요한 필드만 요청한다.

19. 오류 처리

다음 상황을 처리한다.

- 검색된 장소가 없음
- MCP 서버 연결 실패

- Google API 요청 한도 초과
- 이동 경로를 찾을 수 없음
- 예산에 맞는 숙소 없음
- 영업시간과 일정 충돌
- 선택한 날짜에 상품 이용 불가
- LLM JSON 형식 오류

MCP 연결에 실패하더라도 전체 일정 생성이 중단되지 않도록 한다.

예시:

현재 예약 상품 정보를 불러올 수 없어 일반 장소 정보로 일정을 구성했습니다. 상품 정보는 잠시 후 다시 확인해 주세요.

20. 디자인 방향

- 밝고 여행다운 분위기
- 흰색 배경
- 파란색 또는 청록색 포인트
- 둥근 카드
- 장소 사진을 크게 사용
- 일정 정보는 타임라인 형태
- 복잡한 설정은 단계별로 분리
- 지도와 일정을 동시에 확인 가능
- 모바일 우선 반응형 디자인

초안 카드 3개는 단순히 이름만 다르게 하지 말고 시각적으로도 성격이 구분되도록 한다.

예시:

- 관광 집중형: 랜드마크 아이콘
- 감성·맛집형: 음식과 카페 아이콘
- 가성비형: 동전 또는 절약 아이콘

21. 환경변수

```

DATABASE_URL=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

LLM_API_KEY=

MYREALTRIP_API_KEY=
MYREALTRIP_MCP_URL=
TRAVEL_PROVIDER=mock

```

```
GOOGLE_MAPS_BROWSER_API_KEY=  
GOOGLE_MAPS_SERVER_API_KEY=
```

브라우저용 키와 서버용 키를 분리한다.

22. 개발 순서

1단계

- Next.js 프로젝트 생성
- 랜딩 페이지
- 여행 조건 입력 UI
- mock 데이터로 초안 카드 표시

2단계

- LLM 연결
- 사용자 조건을 구조화된 JSON으로 전달
- 초안 3개 생성
- Zod 검증

3단계

- Google 지도 표시
- Google Places 검색
- 장소 마커와 상세 카드 표시

4단계

- Routes API 연결
- 장소 간 이동시간 계산
- 날짜별 지도 경로 표시

5단계

- 마이리얼트립 MCP 연결
- MockMyrealtripProvider를 실제 Provider로 교체
- 예약 상품과 일반 장소 연결

6단계

- AI 수정 채팅
- 일정 잠금
- 부분 수정
- 경로와 예산 재계산

7단계

- Supabase 로그인
- 여행 저장
- 여행 복사

- 공유 링크

한 단계가 정상적으로 동작하고 테스트되기 전에는 다음 단계로 넘어가지 않는다.